

Programmazione a memoria distribuita con MPI

Nei sistemi per il calcolo ad alte prestazioni (HPC), l'architettura hardware detta spesso le regole della programmazione. A differenza dei sistemi a memoria condivisa, nei sistemi a memoria distribuita ogni processo (task) possiede una propria memoria locale e uno spazio di indirizzamento rigorosamente separato. Questo significa che il processo A non può leggere o scrivere direttamente una variabile situata nella RAM del processo B. Per accedere ai dati remoti, il programmatore deve orchestrare attivamente lo scambio di informazioni. Questo approccio si chiama **Message Passing**, e lo standard di fatto per implementarlo è **MPI (Message Passing Interface)**.

▼ Storia e Implementazioni

MPI non è un singolo software, ma una specifica rigorosa redatta dall'MPI Forum, un consorzio nato nel 1994 che riunisce sviluppatori, accademici e vendor hardware. Lo standard definisce le funzioni che i linguaggi supportati (C, C++ e Fortran) devono implementare.

Negli anni, MPI si è evoluto enormemente per adattarsi all'hardware:

- **MPI-1 (1994):** Introdusse le comunicazioni base in un ambiente run-time statico.
- **MPI-2 & MPI-3:** Hanno aggiunto l'I/O scalabile parallelo, ambienti dinamici e operazioni collettive non bloccanti.
- **MPI-5 (2023):** È l'ultima iterazione moderna dello standard.

Essendo una specifica, esistono diverse implementazioni pratiche di MPI:

- **OpenMPI:** Un progetto open-source molto diffuso nato dalla fusione di precedenti tecnologie (come LAM/MPI).
- **MPICH:** Sviluppato dall'Argonne National Laboratory, è un'implementazione libera molto efficiente. L'ultima release (5.0.1) implementa interamente lo standard MPI-5.
- **Intel MPI:** Una versione commerciale ottimizzata specificamente per i processori Intel.

Communicator e Formato dei Messaggi

Quando avvia un programma MPI (ad esempio usando `mpirun`), vengono generati diversi processi indipendenti. Per permettere a questi processi di "parlarsi", MPI utilizza un costrutto logico chiamato **communicator**.

Un communicator è essenzialmente un gruppo chiuso di processi in grado di comunicare tra loro.

- Di default, tutti i processi creati all'avvio vengono inseriti in un communicator globale chiamato `MPI_COMM_WORLD`.
- All'interno di questo gruppo, ogni processo riceve un identificativo numerico univoco chiamato **Rank**, che va da 0 a N-1 (dove N è il numero totale di processi).
- Il processo con Rank 0 è convenzionalmente definito *master* o *root*, ed è solitamente incaricato di coordinare i lavori.

Funzioni:

- `MPI_Init(&argc, &argv)`: inizializza l'ambiente MPI. Deve essere la prima funzione MPI chiamata nel programma.

- `MPI_Comm_size(MPI_COMM_WORLD, &size)` : scrive nella variabile `size` il numero totale di task che fanno parte del communicator specificato.
- `MPI_Comm_rank(MPI_COMM_WORLD, &rank)` : scrive nella variabile `rank` l'identificativo numerico (il "nome") del task chiamante all'interno del communicator.
- `MPI_Get_processor_name(&name, &namelength)` : scrive in `name` il nome fisico dell'host (la macchina) su cui sta girando il task chiamante.
- `MPI_Finalize()` : chiude l'ambiente MPI. Nessuna funzione MPI può essere chiamata dopo questa.

Anatomia di un Messaggio

Quando due processi comunicano, i dati viaggiano all'interno di "pacchetti" logici ben strutturati in due parti:

1. **L'Envelope:** contiene i metadati necessari per il routing, ovvero:
 - *Source*: il rank di chi spedisce.
 - *Destination*: il rank di chi riceve.
 - *Tag*: un intero (scelto dal programmatore) che dà da "etichetta" per identificare la natura del messaggio.
 - *Communicator*: il contesto in cui avviene lo scambio.
2. **Il Body:** contiene il payload:
 - *Type*: il tipo di dato (derivati dai tipi C, es. `MPI_INT` per gli interi, `MPI_FLOAT` per i decimali).
 - *Length*: il numero di elementi contenuti nel buffer.
 - *Buffer*: il puntatore alla locazione di memoria da cui leggere i dati.

Misurare le Prestazioni

Nell'ambito del HPC le funzioni standard del C (come `time()`) non offrono una risoluzione sufficiente per misurare il tempo di esecuzione e scarseggiano di portabilità. Per risolvere questo problema, la libreria fornisce la funzione `MPI_Wtime()`, che restituisce il tempo corrente in secondi a partire da un momento predefinito nel passato, permettendo calcoli di *wall-clock time* estremamente precisi.

Gestione dei Tipi di Dato

Anche se MPI fornisce tipi di base derivati dal C, a volte è necessario inviare strutture eterogenee raggruppandole nel tipo `MPI_PACKED`.

- `MPI_Pack()` : utilizzata per "impacchettare" dati di tipo diverso in un unico buffer contiguo da spedire.
- `MPI_Unpack()` : utilizzata dal ricevente per "scompattare" il buffer ricevuto e recuperare i dati originali.

Comunicazioni: Punto-Punto e Collettive

In MPI lo scambio di dati avviene su infrastrutture di rete molto diverse. Su un cluster HPC, due processi che risiedono sullo stesso nodo fisico useranno la veloce memoria condivisa (SHM) per simulare lo scambio di messaggi. Se risiedono su nodi diversi, i dati viaggeranno attraverso la rete hardware, prediligendo interfacce a bassissima latenza come InfiniBand (IB) o Intel Omnipath, piuttosto che le classiche e lenti reti TCP.

Comunicazioni Punto-Punto (Point-to-Point)

Coinvolgono esattamente due processi: un mittente e un destinatario.

a. APPROCCIO BLOCCANTE

L'esecuzione del task si ferma finché l'operazione di invio o ricezione non raggiunge uno stato di sicurezza per la memoria. I parametri tipici sono il puntatore al buffer (`buf`), il numero di elementi (`count`), il tipo (`datatype`), il destinatario o mittente (`dest` / `source`), l'etichetta del messaggio (`tag`) e il communicator (`comm`).

Le funzioni base sono `MPI_Send()` e `MPI_Recv()`.

- `MPI_Send()` blocca l'esecuzione del codice finché il buffer di invio non è stato svuotato (o copiato in un buffer di sistema) e può essere riutilizzato in sicurezza dal programma.
- `MPI_Recv()` si sblocca solo quando il dato atteso è stato ricevuto interamente e salvato nel buffer. Utile ricordare che il destinatario può usare dei comodini logici (wildcards) come `MPI_ANY_SOURCE` o `MPI_ANY_TAG` per accettare messaggi da chiunque o con qualsiasi etichetta.

Esistono varianti specializzate per il send bloccante:

- `MPI_Ssend` (synchronous): ritorna solo quando il ricevente ha effettivamente iniziato a scaricare i dati.
- `MPI_Bsend` (buffered): ritorna istantaneamente non appena i dati sono copiati in un buffer temporaneo di supporto.
- `MPI_Rsend` (ready): estremamente veloce, ma va usato solo se si ha la certezza assoluta che il ricevente stia già ascoltando (in `Recv`), altrimenti va in errore.
- `MPI_Sendrecv`: esegue una send e una receive contemporaneamente. Si blocca fino a quando il buffer di invio è libero e quello di ricezione è pieno. Fondamentale per evitare i deadlock in schemi di comunicazione circolari.

b. APPROCCIO NON BLOCCANTE

Queste funzioni (dove "I" sta per *immediate*) permettono di mascherare i tempi di rete, ritornando il controllo al programma prima che il trasferimento sia concluso. Richiedono un handle speciale, `&request`, per monitorare lo stato in un secondo momento.

- `MPI_Isend`: ritorna immediatamente senza attendere che i dati vengano copiati nel buffer di rete.
- `MPI_Irecv`: ritorna immediatamente, il programma può fare altri calcoli mentre aspetta l'arrivo del messaggio.
- `MPI_Test`: verifica lo stato di una request (send o receive non bloccante). Imposta la variabile `flag` a 1 se l'operazione è completata, a 0 altrimenti. Non si blocca.
- `MPI_Wait`: funziona come `MPI_Test`, ma ferma l'esecuzione del programma finché la request non è effettivamente terminata (diventando di fatto bloccante in quel punto).

Comunicazioni Collettive e Sincronizzazione

Le operazioni collettive coinvolgono simultaneamente *tutti* i processi appartenenti a un dato Communicator. Se un rank lancia un'istruzione collettiva, nessun altro processo potrà proseguire (causando una barriera implicita) finché l'operazione globale non è conclusa.

Le primitive principali si dividono in:

- **One-to-All:** ad esempio la funzione `MPI_Bcast`, interviene quando un singolo nodo (root) "spara" o copia lo stesso messaggio a tutti gli altri task. Diversamente, `MPI_Scatter` scompone un grosso array del nodo root distribuendone piccoli pezzi (chunk) ai vari processi.
- **All-to-One:** in questo approccio, `MPI_Gather` fa l'opposto dello scatter, raccogliendo l'output distribuito di tutti i rank per ricomporre l'array intero sul nodo root.

- **All-to-All:** primitive come `MPI_Allgather` non solo raccolgono i dati distribuiti, ma ne fanno una copia identica, replicandola per ogni singolo task nel communicator.
- **Operazioni di riduzione:** una categoria di operazioni collettive è la “riduzione” matematico-logica gestita da `MPI_Reduce`. Questa funzione raccoglie tutti i dati da un array distribuito e, durante la fase di ricezione verso il nodo root, applica un’operazione specifica identificata dal parametro `op`. Le operazioni predefinite in C includono `MPI_MAX` e `MPI_MIN` per trovare gli estremi, `MPI_SUM` per le somme e vari operatori logici/bitwise (`MPI_BAND`, `MPI_BOR`, ecc.). Come variante esiste anche `MPI_Allreduce`, che fonde una riduzione seguita da un broadcast del risultato finale a tutti i processi.
- **Sincronizzazione esplicita:** se è necessario obbligare esplicitamente l’intero gruppo di lavoro ad aspettarsi a vicenda prima di proseguire con una certa riga di codice, è possibile utilizzare la funzione `MPI_Barrier`.